

tmx Shell Integration — Operator Guide

Revision: 2 **Last modified:** 2026-07-05T00:00:00Z **Authority:** vasic-digital tmux project **Maintainer:** milosvasic **Scope:** Operator install / uninstall / troubleshoot guide for the `tmx-shell-init.sh` interactive-prompt feature, including the 2026-07-05 wizard + session-password redesign (random-suffix create, existing-session picker, masked single-prompt passwords)

1. Overview

`scripts/tmx-shell-init.sh` is the project-owned shell-init script you source from your `.bashrc` / `.zshrc`. It replaces the hand-pasted snippet that operators previously copied into every rc file (the old snippet drifted silently, blocked SCP / IDE shells with a hung read -r, and forgot the operator's last cwd).

The new init script (2026-07-05 redesign — see the *Interactive session-selection prompt* section below for the full behaviour):

- prompts only on **interactive TTY** shells (skips on SCP, rsync, IDE pipes, cron, non-interactive subshells);
- when the operator presses Enter (blank input) **or types the literal word default** — the two are treated identically — offers a numbered picker of the operator's existing sessions plus 0) None — or, if no sessions exist, defaults to **SKIP** (bare shell, no tmx), exactly as before;
- when the operator types any other, non-blank, non-default name, always creates a **brand-new** session name-NNNN (random 4-digit suffix; `TMX_EXACT_NAME=1` opts out) rather than attaching to an earlier session of the same base name;
- on `tmx new`, the wrapper recalls the session's last cwd via `scripts/tmx-state-bin` and starts the pane there (see [docs/guides/tmx-state.md](#)).

Project authority specs: [docs/superpowers/specs/2026-05-22-tmx-shell-session-resume-design.md](#) §4.A + §5 (original shell-resume feature) and [docs/superpowers/specs/2026-07-05-tmx-wizard-password-redesign-design.md](#) (2026-07-05 wizard + session-password redesign). Password behaviour has its own guide: [docs/guides/tmx-session-passwords.md](#).

Interactive session-selection prompt (2026-07-05 redesign)

When a login shell sources `tmx-shell-init.sh`, the operator sees:

```
[tmx] Enter session name to create (blank = choose existing session):
```

Typing a name — other than the literal word `default` — always creates a **brand-new** session — the real, underlying tmux session name is `<typed-name>-NNNN`, where `NNNN` is a random 4-digit suffix generated fresh on every invocation (so re-typing the same base name later never collides with, or silently reattaches to, an earlier session of the same base name). Set `TMX_EXACT_NAME=1` in the environment to suppress the suffix and use the typed name literally (intended for scripts/automation that need a deterministic name, not for interactive use).

Pressing Enter (blank input) — or typing the literal word `default` (the wizard treats the two identically), if any of the operator's own sessions already exist, shows a numbered menu:

```
[tmx] Existing sessions:
  1) my-session-4821
  2) scratch-0193
  0) None (leave, bare shell)
[tmx] Choose a session to join (0 = none):
```

Choosing a number attaches that session (prompting for its password exactly once if it is password-protected). Choosing `0`, leaving it blank, or an invalid entry after one retry falls through to a bare shell — identical to today's behaviour when no sessions exist yet.

Password prompts

- **A session name that has never been password-protected** (or whose protection was cleared via `tmx delete`): creating it prompts for a password (masked with `*`), then — only if non-blank — a confirmation prompt. A mismatch retries up to 3 times before aborting with no session created and no password set.
- **A session name that already has a persisted password** — whether the underlying tmux server is currently running or was torn down by the idle recycler — is verified **once**. The correct password attaches (or recreates + attaches, if it had been recycled); a wrong password is rejected outright and no session is touched.
- All password input is masked with `*` characters, never shown in plaintext.

2. Prerequisites

- A POSIX shell: bash 3.2 (macOS default), bash 5+, zsh 5.8+, or dash.
- tmx on \$PATH. After bash scripts/setup.sh GREEN, the snippet already prepends scripts/ to PATH for you.
- The Go-built state daemon at scripts/tmx-state-bin (also built by setup.sh). Without it, tmx new still works, the cwd just won't be restored.

3. Installation

3.1 Recommended path (use the generated snippet)

```
cd ~/Projects/tmux # or wherever you cloned this repo  
bash scripts/setup.sh  
# setup.sh GREEN summary prints the exact line to copy.
```

setup.sh writes the snippet into the operator's rc files automatically:

- **Always:** ~/.bashrc and ~/.zshrc (created on Darwin if missing).
- **v1.0.13+:** also ~/.bash_profile and ~/.profile (when present or when \$SHELL is bash). The wrapper invokes the spawned shell with -l (login); bash login shells read .bash_profile NOT .bashrc unless .bash_profile already sources .bashrc (common but not guaranteed). Touching .bash_profile ensures the per-pane PROMPT_COMMAND hook installs reliably — without it, the cwd persist-across-exit feature silently fails.
- **Never:** ~/.zprofile (zsh always sources ~/.zshrc regardless of login vs interactive).

Open a new terminal — that's it.

3.2 Manual path (if you maintain your own rc)

Append the following block to **both** ~/.bashrc and ~/.zshrc, with /Users/milosvasic/Projects/tmux replaced by the absolute path that pwd prints while you stand in the project root:

```
# — vasic-digital optimized tmux  
  
VDIGITAL_TMUX_DIR="/Users/milosvasic/Projects/tmux/scripts"  
if [ -x "$VDIGITAL_TMUX_DIR/tmx" ]; then  
    export PATH="$VDIGITAL_TMUX_DIR:$PATH"  
fi
```

```
[ -r "$VDIGITAL_TMUX_DIR/tmx-shell-init.sh" ] && .
    "$VDIGITAL_TMUX_DIR/tmx-shell-init.sh"
# — end vasic-digital optimized tmux
```

Reload the rc and verify:

```
exec $SHELL -l
which tmx # → /Users/milosvasic/
          Projects/tmux/scripts/tmx
type tmx-shell-init.sh 2>/dev/null # not on PATH itself –
          sourced via `.`
```

3.3 Note for macOS Apple Silicon operators

On macOS the project default cwd is ~/Projects/tmux. If you cloned elsewhere, replace /Users/milosvasic/Projects/tmux everywhere with your absolute path (run pwd while standing in the repo root to read the value).

4. Behavior contract

The init script returns silently (exit 0) without prompting in these situations:

Condition	Why
\$TMUX already set	already inside a tmux session
`[ ! -t 0 ]`	
\$TMX_SKIP non-empty	operator opt-out for this shell only
tmx not on \$PATH	graceful degradation
EOF on stdin (Ctrl-D at prompt)	bare shell
Empty input, or the literal word default, and no sessions exist	bare shell

The script prompts (and acts on the answer) only when none of the above guards trigger. **Empty input, or the literal word default, when sessions DO exist** is not a silent skip — it shows the existing-session picker (see the redesign section above).

Invalid names print an error and return 1:

- length zero or > 64 characters;
- any character outside [A-Za-z0-9_.-].

On a valid **typed** name — other than the literal word default, which is routed to the blank-input branch below — it generates a fresh 4-digit suffix and always creates a brand-new session (2026-07-05 redesign):

```
suffix="$(awk 'BEGIN{srand(); printf "%04d", int(rand()*10000)}')"  
exec sh -c 'exec tmx new -s "$1" tmx-shell-init "${session_name}-  
${suffix}"  
# TMX_EXACT_NAME=1 → use "$session_name" literally, with no  
suffix.'
```

On **blank** input, or the literal word default, it lists the operator's existing sessions and, if any exist, offers a numbered picker (attaching the chosen session via `tmx attach -t <name>`); with no sessions it falls through to a bare shell. `exec` replaces the current shell with the tmux client so that pressing `Ctrl-b d` (detach) returns you cleanly to the parent shell.

5. Worked examples

Example 1 — New interactive login: create a fresh session or pick an existing one

```
$ ssh milosvasic@nezha.local  
# .bashrc is sourced → tmx-shell-init.sh prompts:  
[tmx] Enter session name to create (blank = choose existing  
session): work  
# Typing 'work' ALWAYS creates a brand-new session named work-NNNN  
# (random 4-digit suffix); the pane cwd is restored to your  
# last-recorded pwd for that session name. It never silently  
# reattaches to an earlier 'work'.  
#  
# To attach an EXISTING session instead, press Enter at the  
prompt:  
[tmx] Enter session name to create (blank = choose existing  
session):  
[tmx] Existing sessions:  
1) work-4821  
0) None (leave, bare shell)  
[tmx] Choose a session to join (0 = none): 1  
# → attached to work-4821 (prompted once for its password if  
protected).
```

Example 2 — SCP from another machine never hangs

```
# On your workstation:  
$ scp report.pdf milosvasic@nezha.local:/tmp/  
report.pdf 100% 1.2MB 3.0MB/s  
00:00
```

```
# scp succeeds immediately – tmx-shell-init.sh sees the non-TTY
  stdin
# and silently returns 0 before any prompt is printed.
```

Example 3 — Opt out for this shell only

```
$ TMX_SKIP=1 bash -l
# .bashrc runs, but tmx-shell-init.sh exits early (TMX_SKIP non-
  empty)
# You land directly in a bare bash shell, no tmx, no prompt.
$ unset TMX_SKIP
$ exec $SHELL -l
# Back to the prompt-on-login behaviour.
```

Example 4 — Inside a nested tmux pane

```
$ tmx new -s outer          # creates session 'outer'
# inside the pane:
$ bash -l                  # spawns a sub-shell
# .bashrc runs again, but $TMUX is set, so tmx-shell-init.sh is
  silent.
# No prompt, no nesting – Ctrl-b d still detaches the outer
  session.
```

Example 5 — Manual probe without installing

```
# Run the script under a sub-shell with stdin redirected from /
  dev/null
# (mimics the SCP non-TTY case).
$ bash -c '. /Users/milosvasic/Projects/tmux/scripts/tmx-shell-
  init.sh' </dev/null
$ echo $?
0
# Exit 0 with no output = non-TTY guard fired as designed.
```

6. Troubleshooting

Symptom	Diagnosis	Fix
tmx-shell-init.sh: command not found	The PATH export wasn't loaded (rc edit didn't take)	exec \$SHELL -l to reload, or check the snippet sits BEFORE if [-z "\$PS1"] early-exit guards
Prompt never appears on interactive login	tmx missing from PATH, or shell isn't sourcing your rc Your terminal sends \r\n; the script	Run which tmx; if empty, re-run bash scripts/setup.sh

Symptom	Diagnosis	Fix
Prompt appears, but Enter does nothing	accepts both (POSIX <code>read -r</code>)	Type letters; an empty answer maps to SKIP by design
[tmx] invalid session name '...' on a normal name	The name contains ;, , /, or ..	Rename — only [A-Za-z0-9_.-]{1,64} is accepted
Hangs forever in SCP / rsync	The init script was modified or you're using the legacy hand-pasted snippet	Re-run <code>setup.sh</code> ; the v1.0.9 script tests <code>-t 0</code> BEFORE prompting
Restored cwd is wrong (always <code>\$HOME</code>)	tmx-state-bin not on PATH OR state file <code>~/.tmx/state.json</code> unreadable	<code>ls -la ~/.tmx/state.json</code> ; <code>chmod 600</code> if needed; see tmx-state guide

7. Uninstall

7.1 Standard uninstall (v1.0.11+)

The cleanest path is the dedicated operator entry point:

```
cd ~/Projects/tmux
bash scripts/uninstall.sh
```

This delegates to `setup.sh --uninstall` (single source of truth) and performs ALL of:

1. Strip the — vasic-digital optimized tmux — fenced block from `~/.bashrc` and `~/.zshrc`.
2. Strip any LEGACY pre-v1.0.9 unfenced `if` command `-v tmx ...; fi` snippet (operators who hand-pasted the original snippet).
3. Remove `~/.tmux.conf` (only if generated by `setup.sh`).
4. Remove `scripts/tmx` (generated wrapper).
5. Remove `scripts/tmx-shell-init.sh` (generated v1.0.9+ shell init).
6. Remove `scripts/tmx-state-bin` (v1.0.9+ Go binary).

7.2 Also purge per-session state (optional)

By default the uninstall preserves `~/.tmx/` (per-session last-pwd records) per §9 zero-risk-data-safety. To purge:

```
bash scripts/uninstall.sh --purge-state
```

7.3 Equivalent invocation

```
bash scripts/setup.sh --uninstall           # same as scripts/  
      uninstall.sh  
PURGE_STATE=1 bash scripts/setup.sh --uninstall # same as --  
      purge-state
```

7.4 Clean-slate before reinstall

setup.sh automatically runs the uninstall logic at step 0 (silent) before re-installing — so an operator running `bash scripts/setup.sh` on a host that already had v1.0.9 installed gets a clean state without needing to manually uninstall first. v1.0.11+ behaviour.

7.5 Hand-removal (last resort)

If you do not have access to `setup.sh` or `uninstall.sh` and need to remove the snippet manually:

Open `~/.bashrc` and `~/.zshrc`; delete the block between `# — vasic-digital optimized tmux —` and `# — end vasic-digital optimized tmux —`. Then `rm scripts/tmx scripts/tmx-shell-init.sh scripts/tmx-state-bin ~/.tmux.conf` (only the last one if you didn't keep your own; check for vasic-digital optimized tmux configuration marker in the file first).

8. Cross-references

- [docs/guides/tmx-state.md](#) — Go state daemon CLI + state file schema
- [docs/guides/tmx-ssh-dispatch.md](#) — `ssh host-tmx <session> setup`
- [docs/manual/tmx-shell-integration.md](#) — end-user master manual
- [docs/scripts/tmx-shell-init.md](#) — §11.4.18 script companion
- Spec: [docs/superpowers/specs/2026-05-22-tmx-shell-session-resume-design.md](#) §4.A + §5

9. Anti-bluff (§11.4)

This feature is covered by Layer 3 tests `28_default_skip.sh`, `29_default_skip_blank.sh`, `30_non_tty_skip.sh`, `35_session_name_validation.sh`, `37_nested_tmux_skip.sh`, and the Layer 4 paired mutation M20 (strips the `-t 0` guard, asserts test

30 FAILs). Every PASS captures positive runtime evidence (no tmux process spawned in skip-paths; named-session created in valid-name paths).

10. Last verified

2026-05-22 on Darwin arm64 (macOS 15.x, bash 3.2 / zsh 5.9, tmux 3.6a) with bash scripts/setup.sh --verify-only GREEN.